

📘 07차시 — 파인튜닝 학습데이터 전처리

교육시간: 2일차 첫 시간 (60분)
실습 모델: Qwen/Qwen2.5-0.5B-Instruct (제작 선택; 본 차시에서는 다운로드하지 않음)
검증 기준일: 2026-11-04

01 | 개요·60분 진행

07차시 — 파인튜닝 데이터셋 전처리. 2일차 첫 시간(60분). 06차시에서는 모델의 학습 전후 평가 조건을 다뤘다. 이번에는 08차시 SFTTrainer에 넣을 대화형 데이터를 만든다.

0~10분: 원본 구조와 누출 위험 / 10~20분: 필드 검사·정규화 / 20~40분: 전처리 코드 실행 / 40~50분: 분할 결과 검증 / 50~60분: 평가 및 오류 복구.

산출물: train.jsonl, validation.jsonl, rejected.json, report.json. 모델 학습은 이 차시에 수행하지 않는다.

---

02 | 선수지식·데이터 규약

원본 JSONL은 한 줄에 하나의 JSON 객체를 기록한다. 필수 키 instruction(질문), response(답변)는 모두 문자열이어야 한다.

잘못된 데이터 사례: 빈 질문, 비문자열 값, 깨진 JSON, 동일 질문의 반복, 지나치게 긴 답변.

SFT(Supervised Fine-Tuning, 지도 미세조정)는 질문과 정답 예시를 학습한다. 검증셋은 학습 중 가중치 갱신에 사용하지 않는다. 개인정보·저작권·출처·라이선스 검토는 데이터 품질 검사의 별도 필수 절차이다.

03 | 핵심 원리·데이터 누출

정규화 → 중복 제거 → 분할 순서가 중요하다. 먼저 분할하고 중복을 제거하면 동일한 질문이 양쪽에 들어갈 수 있다.

동일 질문에 답변만 다르게 붙은 행도 질문 키 기준으로 제거한다. 공백과 대소문자를 정규화해 단순 중복을 줄인다. 의미가 같은 다른 표현까지 탐지하는 의미 중복 검사는 별도 확장 과제이다.

고정 난수 시드 42는 재현성을 돕지만 데이터셋 자체가 바뀌면 결과도 바뀐다. 원본 SHA-256을 기록해 버전 차이를 추적한다.

04 | 처리 흐름·구조도

[원본 JSONL] → [파싱/필수 필드 검증] → [공백 정리·길이 검사] → [질문 키 중복 제거] → [시드 고정 셔플] → [학습/검증 분할] → [messages 변환] → [JSONL 저장]

└→ [거부 사유 기록]

학습·검증 데이터는 서로 겹치지 않아야 한다. 검증 파일을 다음 차시의 평가·학습 검증용으로 사용한다. 동일 문서에서 파생된 여러 문항은 실제 프로젝트에서는 문서 단위로 그룹 분할해야 한다.

05 | Ubuntu 준비·명령어

작업 디렉터리: lab_py/lesson07 또는 이 ZIP을 푼 폴더. Python 3.11 표준 라이브러리만으로 1차 실습 가능하다.

python3.11 --version # 파이썬 버전 확인

python3.11 -m venv .venv # 가상환경 생성

source .venv/bin/activate # 가상환경 활성화

python lesson07_preprocess.py --input lesson07_raw.jsonl --output processed # 데이터 정제 및 분할

추가 실습: pip install datasets transformers # Hugging Face 라이브러리 설치(인터넷 필요). VS Code에서 .venv 인터프리터를 선택한다.

---

06 | 실습 코드·핵심 구문

lesson07_preprocess.py의 build()는 줄별 JSON 파싱과 validate()를 수행한다. 잘못된 행은 rejected.json에 기록한다.

normalized()는 연속 공백을 하나로 합치고 casefold()로 영문 대소문자를 정규화한다. seen 집합에 질문 키가 있으면 해당 행을 중복으로 거부한다.

random.Random(seed).shuffle(clean)으로 결과를 재현할 수 있게 섞는다. 검증 비율 0.2를 적용하고, 최소 1개 이상의 학습·검증 샘플을 유지한다.

변환 예시: {"messages": [{"role":"user","content":"질문"},{"role":"assistant","content":"답변"}]}

07 | 코드 검증·출력 해석

예상 결과(동봉 예제 15줄): 유효 고유 질문 12개, 중복 1개, 빈 질문 1개, JSON 오류 1개. train 10개, validation 2개, rejected 3개, overlap 0개. 실제 출력은 실행해서 확인한다.

report.json의 sha256은 입력 데이터 파일의 내용 식별자이며 품질 점수가 아니다. rejected.json은 행 번호와 오류 원인을 포함한다.

assert train_keys.isdisjoint(val_keys)는 질문 문자열의 교집합이 없음을 검사한다. 의미 중복이나 외부 평가셋 누출까지 보장하지 않는다.

08 | 따라하기 실습·Hugging Face 연결

① 원본 파일을 열고 잘못된 세 줄을 찾는다. ② 전처리 코드를 실행한다. ③ processed/train.jsonl과 validation.jsonl을 비교한다. ④ report.json의 overlap 값을 확인한다.

추가 라이브러리 확인 코드:

```
from datasets import load_dataset

ds = load_dataset("json", data_files={"train":"processed/train.jsonl", "validation":"processed/validation.jsonl"})

print(ds["train"][0]["messages"])
```

모델 토크나이저 사용 시 tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=False)로 학습용 대화를 직렬화한다. 학습용 전체 대화에는 일반적으로 생성 시작 프롬프트를 추가하지 않는다.

---

09 | 오류 점검·확장 과제

JSONDecodeError: 원본 파일의 따옴표·쉼표·줄 단위 구조를 점검한다. UnicodeDecodeError: UTF-8로 저장한다. ValueError(샘플 부족): 유효 고유 질문을 2개 이상 확보한다. ModuleNotFoundError: datasets 실습만 선택적으로 설치한다.

확장 A: 개인정보 패턴을 발견하면 자동 삭제하지 말고 검토 대기 목록으로 분리한다. 확장 B: 문서 ID별 그룹 분할로 같은 원문에서 나온 문항이 양쪽에 걸치지 않도록 한다. 확장 C: 실제 토큰 길이 제한은 문자 수가 아니라 토크나이저 토큰 수로 판단한다.

10 | 평가·체크리스트·참고

체크: (1) 유효·거부 행 수 합이 원본 행 수와 일치하는가? (2) 학습·검증 질문의 교집합은 0인가? (3) 두 파일의 messages 역할 순서는 올바른가? (4) 동일 시드와 동일 원본에서 같은 결과가 나오는가? (5) 검증 데이터를 학습에 섞지 않았는가?

실습 제출: processed 폴더 4개 파일과 실행 터미널 캡처. 실제 검증과 예상 출력은 구분한다.

공식 출처(2026-11-04 확인): https://huggingface.co/docs/datasets/loading ; https://huggingface.co/docs/datasets/process ; https://huggingface.co/docs/transformers/main/chat_templating

08차시: SFTTrainer에서 대화형 JSONL을 불러오고 학습 형식을 점검한다.

전체 Python 소스

```
"""07차시: 외부 모델 다운로드 없이 SFT 데이터 검증 · 정제 · 분할."""
import argparse
import hashlib
import json
import random
import re
from pathlib import Path


def normalized(text):
    # 공백 차이만 있는 중복 질문도 같은 질문으로 취급한다.
    return re.sub(r"\s+", " ", text.strip()).casefold()


def validate(row):
    # 최소 필드와 데이터형 검사: 임의의 비문자열은 거부한다.
    if not isinstance(row, dict):
        return None, "객체가 아님"
    if not isinstance(row.get("instruction"), str) or not isinstance(row.get("response"), str):
        return None, "필수 문자열 누락"
    q, a = row["instruction"].strip(), row["response"].strip()
    if not q or not a:
        return None, "빈 질문/답변"
    if len(q) > 400 or len(a) > 1200:
        return None, "길이 제한 초과"
    # 실무에서는 비밀정보 · 개인정보 탐지 규칙을 별도로 확장해야 한다.
    return {"instruction": q, "response": a}, None


def build(input_path, out_dir, seed=42, ratio=0.2):
    if not 0 < ratio < 1:
        raise ValueError("검증 비율은 0과 1 사이여야 합니다")
    out_dir.mkdir(parents=True, exist_ok=True)
    seen, clean, rejected = set(), [], []
    with input_path.open(encoding="utf-8") as f:
        for line_no, line in enumerate(f, 1):
            if not line.strip():
                continue
            try:
                row = json.loads(line)
            except json.JSONDecodeError:
                rejected.append({"line": line_no, "reason": "JSON 오류"})
                continue
            item, reason = validate(row)
            if reason:
                rejected.append({"line": line_no, "reason": reason})
                continue
            key = normalized(item["instruction"])
            if key in seen:
                rejected.append({"line": line_no, "reason": "질문 중복"})
                continue
            seen.add(key)
            clean.append(item)
    if len(clean) < 2:
        raise ValueError("유효한 고유 질문이 최소 2개 필요합니다")
    rng = random.Random(seed)
    rng.shuffle(clean)
    n_val = max(1, min(len(clean)-1, round(len(clean)*ratio)))
    val, train = clean[:n_val], clean[n_val:]
    def write(name, rows):
        path = out_dir/name
        with path.open("w", encoding="utf-8") as f:
            for row in rows:
```

```
                # SFTTrainer의 대화형 데이터 형식으로 변환한다.
                messages = [{"role": "user", "content": row["instruction"]},
                            {"role": "assistant", "content": row["response"]}]
                f.write(json.dumps({"messages": messages}, ensure_ascii=False)+"\n")
    write("train.jsonl", train)
    write("validation.jsonl", val)
    (out_dir/"rejected.json").write_text(json.dumps(rejected, ensure_ascii=False, indent=2), encoding="utf-8")
    train_keys = {normalized(r["instruction"]) for r in train}
    val_keys = {normalized(r["instruction"]) for r in val}
    assert train_keys.isdisjoint(val_keys), "질문 단위 데이터 누출"
    report = {"input": str(input_path), "seed": seed, "train": len(train),
              "validation": len(val), "rejected": len(rejected),
              "overlap": len(train_keys & val_keys),
              "sha256": hashlib.sha256(input_path.read_bytes()).hexdigest()}
    (out_dir/"report.json").write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False, indent=2))
    return report


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--input", type=Path, default=Path(__file__).with_name("lesson07_raw.jsonl"))
    parser.add_argument("--output", type=Path, default=Path(__file__).with_name("processed"))
    args = parser.parse_args()
    build(args.input, args.output)
```

적대적 검증·개선 기록 (2026-11-05)

반영: 전처리 결과 train=10, validation=2의 실제 JSONL 파일을 통합 패키지에 복구했습니다.

검증 구분: 코드 구문·입력 데이터·PDF 및 HTML 생성과 검증은 수행했으나 실제 모델 다운로드·GPU 학습·추론 품질·Ollama 배포는 별도의 실물 검증이 필요합니다.

재현성: 교육용 5스텝 학습의 손실 감소나 정확도 향상을 보장하지 않습니다. 모델 ID, 데이터 버전, 장치·패키지 버전, 난수 시드, 생성 설정을 기록해야 합니다.

공식 근거: SFTTrainer, PEFT 연동, 템플릿, PEFT 모델 저장, 양자화, Ollama GGUF (2026-11-05 확인)